Utsav Prajapati
DevOps Engineer

Project Documentation

Central Monitoring System for Kubernetes

1. Project Objective

- To deploy a comprehensive, real-time monitoring system for a **3-node Kubernetes cluster** using the **Prometheus Community Helm chart** and visualize metrics with Grafana.

2. Problem Solved

- With applications running on a Kubernetes cluster, it's difficult to track the performance and health of individual pods and nodes without a proper monitoring tool.
- This lack of visibility makes it hard to troubleshoot issues, manage resources effectively, and proactively identify problems before they cause outages.

3. My Role & Responsibilities (Solo Project)

- **Cluster Setup:** Worked with a pre-existing 3-node Kubernetes cluster

  (1 master, 2 worker nodes).

- **Prometheus Deployment:** Deployed the entire Prometheus monitoring stack (including Alertmanager, Node Exporter) into a dedicated monitoring namespace using a single **Helm chart**.
- **Service Exposure:** Configured the Prometheus server to be accessible outside the cluster using a **NodePort** service for easy access.
- **Grafana Integration:** Configured Grafana to use the deployed Prometheus instance as its primary data source.
- **Dashboard Implementation:** Imported a powerful, pre-built Kubernetes dashboard from a public repository to immediately visualize cluster-wide metrics.

4. Technology Stack

- **Container Orchestration:** Kubernetes (1 Master, 2 Worker Nodes)
- **Deployment Tool:** Helm
- **Monitoring Core:** Prometheus
- **Visualization:** Grafana

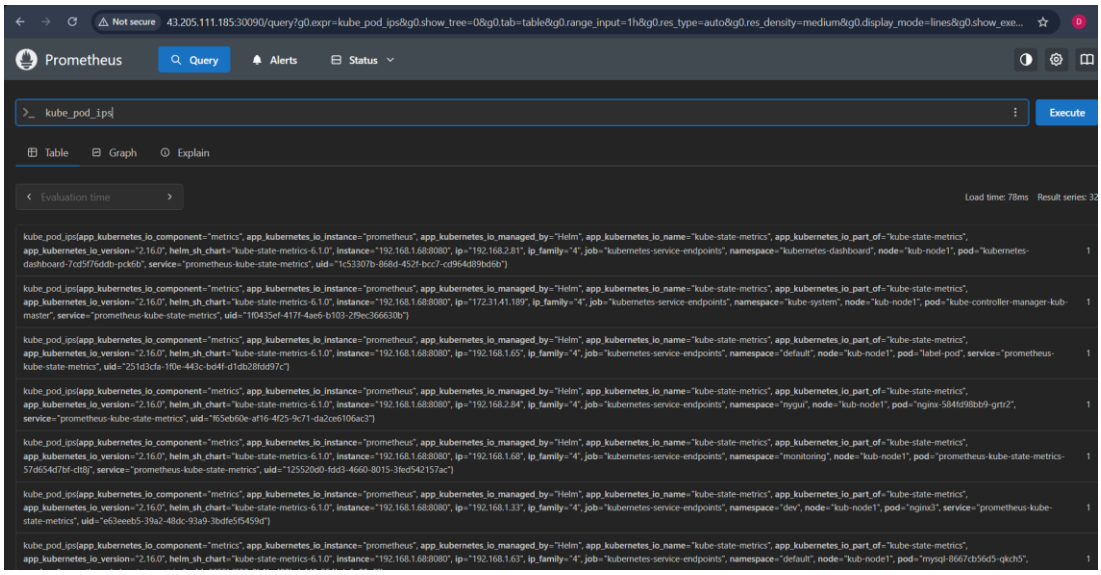
5. Implementation Workflow

- **1. Namespace Creation:** First, I created a dedicated monitoring namespace to keep all monitoring components isolated and organized.

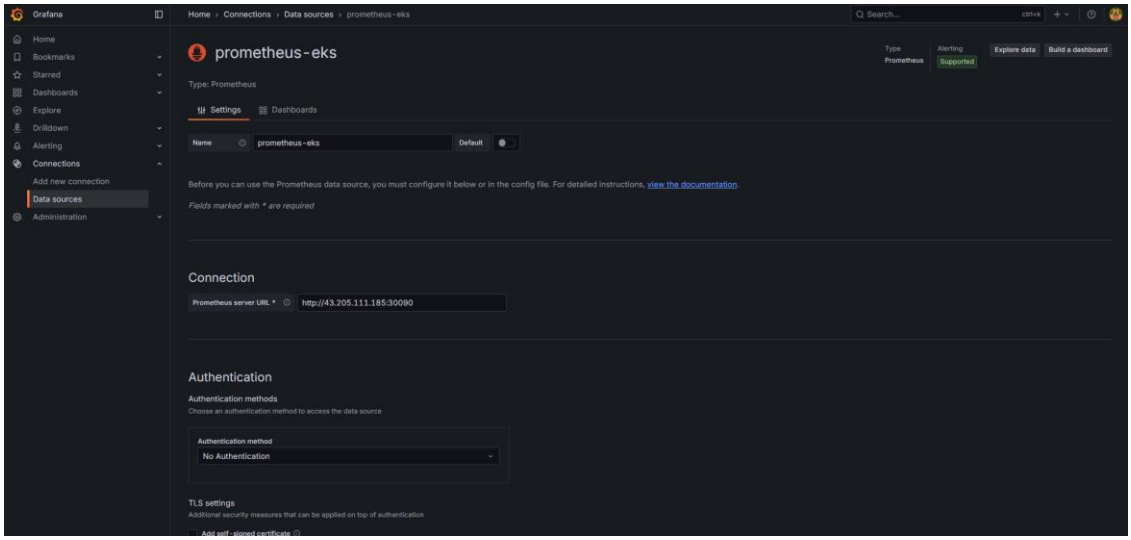
- **2. Helm Chart Deployment:** I used Helm to deploy the prometheus-community/prometheus chart. This single command automatically set up Prometheus, Node Exporter on all nodes. Exposed the Prometheus server on Port 30090.

```
root@kub-master: ~
root@kub-master:~# kubectl get nodes
NAME         STATUS   ROLES           AGE   VERSION
kub-master   Ready    control-plane   19d   v1.33.2
kub-node1    Ready    <none>          19d   v1.33.2
kub-node2    Ready    <none>          19d   v1.33.2
root@kub-master:~# kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
byreplica-898st          1/1     Running   0          10d
byreplica-dgzjn          1/1     Running   0          10d
byreplica-j6pcw          1/1     Running   0          10d
byreplica-nztcp          1/1     Running   0          10d
byreplica-qqgbm          1/1     Running   0          10d
label-pod                1/1     Running   0          10d
mysql-8667cb56d5-qkch5   1/1     Running   0          11d
root@kub-master:~# kubectl get pods -n monitoring
NAME                                             READY   STATUS    RESTARTS   AGE
prometheus-alertmanager-0                        0/1     Pending   0          4d23h
prometheus-kube-state-metrics-57d654d7bf-clt8j   1/1     Running   0          4d23h
prometheus-prometheus-node-exporter-m6h7g        1/1     Running   0          4d23h
prometheus-prometheus-node-exporter-mtdpc        1/1     Running   0          4d23h
prometheus-prometheus-node-exporter-w9pqx        1/1     Running   0          4d23h
prometheus-prometheus-pushgateway-784c485d55-l5sfl   1/1     Running   0          4d23h
prometheus-server-78cc6584ff-594qw               2/2     Running   0          4d23h
root@kub-master:~#
```

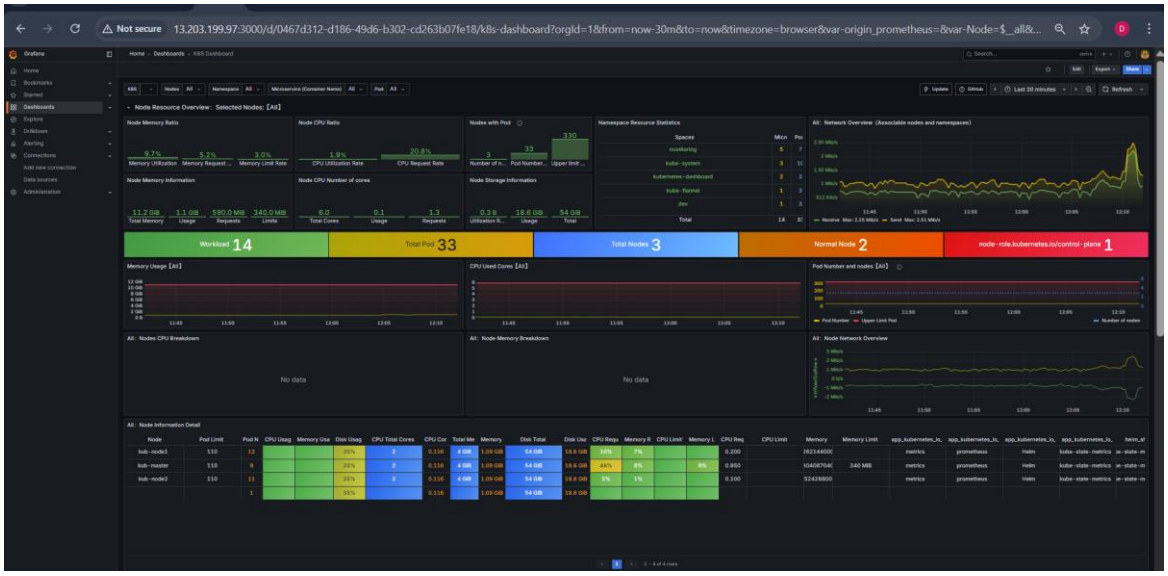
- **3. Accessing Prometheus:** I verified the deployment by accessing the Prometheus UI through the URL.



- **4. Grafana Data Source Configuration:** In Grafana, I added a new data source, selected Prometheus, and pointed it to the internal Kubernetes service URL for the Prometheus server.



- **5. Importing the Dashboard:** I imported a comprehensive Kubernetes monitoring dashboard from a community source, which instantly provided visualizations for nodes, pods, and overall cluster resource usage.



6. Core Implementation Commands

- **Helm Chart Installation:** These are the exact commands used to set up the entire monitoring stack.

```
# 1. Create a dedicated namespace
kubectl create namespace monitoring

# 2. Add the Prometheus community repository
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts

# 3. Update the repository
helm repo update

# 4. Install the Prometheus chart with custom values
helm install prometheus prometheus-community/prometheus \
  --set alertmanager.persistentVolume.enabled=false \
  --set server.persistentVolume.enabled=false \
  --set server.service.type=NodePort \
  --set server.service.nodePort=30090 -n monitoring
```

7. Challenges & Solutions

- **Challenge:** Setting up each monitoring component (Prometheus, Node Exporter, Alertmanager) manually would be complex and time-consuming.
- **Solution:** I leveraged the official `prometheus-community` **Helm chart**. This abstracted away the complexity, allowing me to deploy the entire stack with a single, repeatable command. I used `--set` flags to customize the deployment to my specific needs (like using a `NodePort` service and disabling persistence).

8. Project Outcomes

- **Result:** A fully functional, centralized monitoring system for the Kubernetes cluster, deployed efficiently and quickly.
- **Impact:** Provided immediate, deep visibility into node health, pod status, and resource consumption, enabling faster troubleshooting and better resource management.
- **Key Learning:** Gained practical experience deploying complex applications on Kubernetes using Helm. Learned how to customize Helm releases and integrate standard tools like Prometheus and Grafana in a cloud-native environment.